

Guião 05 – Cluster Kubernetes em VMs Proxmox

Infraestruturas de Computação na Nuvem, Aula 05

1 Objetivos

- Provisionar, a partir do Proxmox, as VMs que constituirão um cluster Kubernetes real com múltiplos nós.
- Inicializar o control plane com `kubeadm` e juntar um worker node ao cluster.
- Instalar um plugin de rede CNI e compreender o seu papel.
- Criar um Deployment e um Service a partir de manifestos YAML declarativos.
- Observar o agendamento de Pods entre nós e o mecanismo de self-healing.

2 Pré-requisitos e Material

- Acesso ao Proxmox VE do servidor central e ao intervalo de VMIDs atribuído.
- Duas VMs Linux (Ubuntu Server LTS), criadas para este efeito: `k8s-cp` (control plane) e `k8s-w1` (worker). Recomenda-se 2 vCPU e 2 GB de RAM em cada, no mínimo.
- As duas VMs devem estar na mesma rede (mesma bridge do Proxmox) e alcançar-se mutuamente.
- Acesso SSH a ambas as VMs, com privilégios de `sudo`.

Nota sobre recursos: este guião cria duas VMs por aluno ou por grupo. Num servidor partilhado por uma turma inteira, isto pode esgotar a memória disponível. Confirme com a docente o modelo a adotar: por grupo (recomendado), ou individualmente com um número reduzido de nós. A alternativa mais leve consiste em cada grupo montar um cluster de dois nós, e partilhar observações entre grupos.

3 Introdução

Na Aula 04 corremos contentores num único host, com Docker. O passo seguinte – e o tema desta aula – é a orquestração: distribuir contentores por vários hosts, reagir automaticamente a falhas e expor serviços de forma estável.

Ao contrário de uma instalação de aprendizagem num único nó, aqui vamos construir um cluster **real, com dois nós**, sobre VMs provisionadas no Proxmox. Isto permite observar coisas que um cluster de nó único não mostra: o *scheduler* a decidir em que nó colocar cada Pod, a rede entre nós assegurada por um plugin CNI, e a diferença entre control plane e worker node.

A ferramenta usada é o `kubeadm`, o instalador oficial do projeto Kubernetes para criar clusters conformes.

4 Parte 1 – Provisionar as VMs no Proxmox

1. **Criar as duas VMs.** A partir da interface web do Proxmox (ou por CLI, como no Guião 03), crie duas VMs Ubuntu Server. Se tiver criado um template no Guião 03, clone-o – é bastante mais rápido:

```
$ qm clone <id-template> <vmid-cp> --name k8s-cp --full
$ qm clone <id-template> <vmid-w1> --name k8s-w1 --full
$ qm set <vmid-cp> --memory 2048 --cores 2
$ qm set <vmid-w1> --memory 2048 --cores 2
$ qm start <vmid-cp> && qm start <vmid-w1>
```

2. **Definir hostnames distintos.** Em cada VM, via SSH:

```
# Na VM do control plane:
$ sudo hostnamectl set-hostname k8s-cp

# Na VM do worker:
$ sudo hostnamectl set-hostname k8s-w1
```

Se as VMs foram clonadas do mesmo template, confirme também que têm endereços MAC e machine-id distintos (o Proxmox gera novos MACs ao clonar; para o machine-id, se necessário: `sudo rm -f /etc/machine-id && sudo systemd-machine-id-setup`).

3. **Registar os endereços IP** de ambas as VMs – serão necessários adiante:

```
$ ip addr show
```

5 Parte 2 – Preparar ambos os nós

Os passos seguintes executam-se **em ambas as VMs**.

4. **Desativar a swap.** O kubelet exige que a swap esteja desativada:

```
$ sudo swapoff -a
$ sudo sed -i 's/ swap / s/^/#/' /etc/fstab
```

5. **Carregar os módulos de kernel necessários:**

```
$ cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
$ sudo modprobe overlay
$ sudo modprobe br_netfilter
```

6. **Configurar os parâmetros de rede** para que o tráfego em bridge seja visto pelo iptables:

```
$ cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF
$ sudo sysctl --system
```

7. **Instalar o runtime de contentores (containerd).** Note que é o mesmo containerd referido na arquitetura Docker da Aula 04:

```
$ sudo apt update
$ sudo apt install -y containerd
$ sudo mkdir -p /etc/containerd
$ containerd config default | sudo tee /etc/containerd/config.toml > /dev/null
```

Ative o driver de cgroups `systemd`, requisito para o kubelet funcionar corretamente:

```
$ sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/
  config.toml
$ sudo systemctl restart containerd
$ sudo systemctl enable containerd
```

8. **Instalar kubeadm, kubelet e kubectl** a partir do repositório oficial. Confirme com a docente a versão a usar; o exemplo seguinte assume a série v1.30 (ajuste se necessário):

```
$ sudo apt install -y apt-transport-https ca-certificates curl gpg
$ curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key \
  | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
$ echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs
  .k8s.io/core:/stable:/v1.30/deb/ /' \
  | sudo tee /etc/apt/sources.list.d/kubernetes.list
$ sudo apt update
$ sudo apt install -y kubelet kubeadm kubectl
$ sudo apt-mark hold kubelet kubeadm kubectl
```

O `apt-mark hold` impede atualizações automáticas que poderiam quebrar o cluster.

6 Parte 3 – Inicializar o cluster

9. **Inicializar o control plane.** *Apenas na VM k8s-cp:*

```
$ sudo kubeadm init --pod-network-cidr=10.244.0.0/16
```

O valor de `--pod-network-cidr` corresponde ao intervalo esperado pelo Flannel, o plugin CNI que instalaremos a seguir. **Guarde o comando `kubeadm join` apresentado no final** – será necessário no passo seguinte.

10. **Configurar o kubectl** para o utilizador normal, ainda em `k8s-cp`:

```
$ mkdir -p $HOME/.kube
$ sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
$ sudo chown $(id -u):$(id -g) $HOME/.kube/config
$ kubectl get nodes
```

O nó aparecerá em estado `NotReady` – falta a rede.

11. **Observar porque o nó não está pronto.** Confirme que os Pods de sistema relacionados com DNS estão pendentes:

```
$ kubectl get pods -n kube-system
```

Os Pods do CoreDNS ficam em `Pending` porque ainda não existe rede de Pods. Isto demonstra na prática o papel do CNI discutido na teoria.

12. **Instalar o plugin CNI (Flannel):**

```
$ kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
```

Aguarde alguns instantes e volte a verificar:

```
$ kubectl get nodes
$ kubectl get pods -n kube-system
```

O nó deve agora passar a Ready e o CoreDNS a Running.

13. **Juntar o worker ao cluster.** Na VM *k8s-w1*, execute o comando `kubeadm join` guardado anteriormente:

```
$ sudo kubeadm join <IP-do-cp>:6443 --token <token> \
--discovery-token-ca-cert-hash sha256:<hash>
```

Se tiver perdido o comando, pode gerá-lo de novo a partir do control plane:

```
$ kubeadm token create --print-join-command
```

14. **Confirmar o cluster completo**, a partir de *k8s-cp*:

```
$ kubectl get nodes -o wide
```

Devem aparecer dois nós em estado Ready, com papéis (ROLES) distintos.

15. **Inspecionar os componentes do control plane.** Identifique os componentes estudados na teoria a correr como Pods:

```
$ kubectl get pods -n kube-system -o wide
```

Localize o kube-apiserver, o etcd, o kube-scheduler e o kube-controller-manager, e repare em que nó cada um corre.

7 Parte 4 – Implantar uma aplicação

16. **Criar o manifesto do Deployment**, `deployment.yaml`, com pedidos e limites de recursos:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  labels:
    app: web-app
spec:
  replicas: 4
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
        - name: web-app
```

```

image: nginx:1.25
ports:
  - containerPort: 80
resources:
  requests:
    memory: "64Mi"
    cpu: "100m"
  limits:
    memory: "128Mi"
    cpu: "250m"
readinessProbe:
  httpGet:
    path: /
    port: 80
  initialDelaySeconds: 3
  periodSeconds: 5

```

17. Aplicar o manifesto e observar a distribuição pelos nós:

```

$ kubectl apply -f deployment.yaml
$ kubectl get deployments
$ kubectl get replicaset
$ kubectl get pods -o wide

```

A coluna NODE mostra em que nó o scheduler colocou cada Pod. Registe a distribuição obtida.

18. Criar o Service, service.yaml:

```

apiVersion: v1
kind: Service
metadata:
  name: web-app-service
spec:
  selector:
    app: web-app
  ports:
    - port: 80
      targetPort: 80
  type: NodePort

```

19. Aplicar o Service e aceder à aplicação:

```

$ kubectl apply -f service.yaml
$ kubectl get services

```

Identifique a porta atribuída (no intervalo 30000–32767) e aceda através de *qualquer* um dos nós:

```

$ curl http://<IP-do-k8s-cp>:<nodeport>
$ curl http://<IP-do-k8s-w1>:<nodeport>

```

Confirme que ambos respondem – é o kube-proxy a encaminhar o tráfego, independentemente do nó onde os Pods realmente estão.

20. Testar a resolução de nomes interna (CoreDNS). A partir de um Pod temporário dentro do cluster:

```
$ kubectl run teste --rm -it --image=busybox:1.36 --restart=Never -- sh
/ # wget -O- http://web-app-service
/ # exit
```

8 Parte 5 – Escalar e observar o self-healing

21. **Escalar o número de réplicas** e observar onde são colocadas as novas:

```
$ kubectl scale deployment web-app --replicas=6
$ kubectl get pods -o wide
```

22. **Apagar um Pod manualmente e observar a reconciliação:**

```
$ kubectl get pods -l app=web-app
$ kubectl delete pod <nome-de-um-pod>
$ kubectl get pods -l app=web-app --watch
```

Observe que, poucos segundos depois, o ReplicaSet cria um novo Pod. Interrompa com Ctrl+C quando confirmar.

23. **Simular a falha de um nó inteiro.** Este é o teste que um cluster de nó único não permite fazer. A partir do Proxmox, pare abruptamente a VM do worker:

```
$ qm stop <vmid-w1>
```

A partir de k8s-cp, observe a reação do cluster ao longo de alguns minutos:

```
$ kubectl get nodes --watch
$ kubectl get pods -o wide
```

O nó passa a NotReady e, decorrido o tempo de tolerância configurado, os Pods que lá corriam são recriados no nó restante. Registe quanto tempo demorou até observar a recriação.

24. **Recuperar o nó** e confirmar que volta a integrar o cluster:

```
$ qm start <vmid-w1>
$ kubectl get nodes
```

25. **Limpeza.** Remova os recursos aplicados. Mantenha o cluster montado se for reutilizado nas Aulas 08 e 09 (confirmar com a docente); caso contrário, pare também as VMs:

```
$ kubectl delete -f service.yaml
$ kubectl delete -f deployment.yaml
```

9 Entregável / Questões de Verificação

- Submeta os ficheiros `deployment.yaml` e `service.yaml`, e a saída de `kubectl get nodes -o wide` com os dois nós em Ready.
- Antes de instalar o Flannel, o nó estava NotReady e o CoreDNS em Pending. Explique porquê, com base no papel do plugin CNI no modelo de rede do Kubernetes.
- Registe a distribuição de Pods pelos nós (`kubectl get pods -o wide`). Que componente do control plane tomou essa decisão e que informação usou?

- d) Acedeu à aplicação através do IP de ambos os nós, mesmo não havendo Pods em todos eles. Que componente torna isto possível?
- e) Quanto tempo decorreu entre parar a VM do worker e a recriação dos seus Pods no outro nó? Relacione o comportamento observado com o ciclo de reconciliação estudado na teoria.
- f) Qual a diferença de responsabilidades entre o Deployment e o ReplicaSet que ele cria?
- g) Definiu `requests` e `limits` no manifesto. Qual a diferença entre os dois, e através de que mecanismo do kernel (visto na Aula 04) são os limites efetivamente aplicados?
- h) Este cluster corre sobre VMs que, por sua vez, correm sobre o Proxmox. Que vantagem tem esta arquitetura em relação a instalar o Kubernetes diretamente no servidor físico?